AI-Agent-Skills vs. VidiSmart Agent Army — Gap Analysis

Date: 2026-09-02
Source reference: `reference/ai-agent-skills/` (shallow clone of `github.com/sreerevanth/AI-Agent-Skills`, `.git` stripped to a plain snapshot; code MIT, prose CC BY 4.0).
Comparison baseline (implemented agent army): `agents/*`, `agent-backend/`, `agent-control/`, `agents-live.json`, `mcp-servers/`, `plans/AI-AGENT-PROTOCOL.md`, `plans/PROJECT-SAFEGUARDS-MANDATORY.md`, `plans/VIDISMART_VECTOR_STACK_ARCHITECTURE.md`, `agentarmy.html`.
Method: Every row of `reference/ai-agent-skills/SKILL_CATALOG.md` was compared against the baseline. Legend below; every ØÝ5 cites the specific repo page it came from.

Legend

- 'Implemented — a VidiSmart file/feature concretely covers the repo concept.
- Ø=ßá Partial — the concept exists (spec, plan, or shallow/narrow implementation) but is shallower than the repo's treatment.
- ØÝ5 New idea / not implemented — candidate to adopt; repo page path cited.

> Note on agent specs: agents/research-agent.md, agents/content-creation-agent.md, and agents/visual-design-agent.md are aspirational spec documents (from the "Accelerating at the Speed of AI" book project), not confirmed running code. Items that exist only as these specs are rated Ø=ßá, not ' .

' Already implemented

Repo concept	VidiSmart coverage
File System Access (02-tool-use/README.md#file-system-access)	agents/verification-agent.ts recursively scans the filesystem; agents/text-rebranding-agent.ts modifies files. Real, running file access.
Structured Outputs (03-communication/README.md#structured-outputs)	agents/base-agent.ts defines typed AgentResult / ChangeRecord / ScanResult interfaces; agents/hermes_bridge.py emits structured JSON; agents-live.json is a schema'd feed; research/content specs define JSON output schemas.
Guardrails (07-safety-alignment/README.md#guardrails)	plans/AI-AGENT-PROTOCOL.md + plans/PROJECT-SAFEGUARDS-MANDATORY.md: hard-protected file list, zero-tolerance blocked commands, layered pre-operation checklist.
Human-in-the-Loop Approval (07-safety-alignment/README.md#human-approval)	Same protocol docs mandate backup-first, authorization confirmation, and STOP-on-red-flag before consequential/destructive operations.
Telemetry & Metrics (14-observability/README.md#telemetry)	agent-backend/fleet-schema.sql !' fleet_telemetry table (tokens_per_sec, memory_mb, cpu_percent, runtime_sec, cost_usd).
Logging (14-observability/README.md#logging)	agents/hermes_bridge.py structured JSON progress logs; BaseAgent.log()/error(); fleet_activity event table.
Vector Databases (10-rag/vector-databases.md)	plans/VIDISMART_VECTOR_STACK_ARCHITECTURE.md: Vespa (Vidi Ai) running on :8089 as primary vector DB.
Docker for Agents (16-deployment/README.md#docker)	agent-backend/Dockerfile + docker-compose.yml; agent-control/Dockerfile + docker-compose.yml.

Ø=ßá Partially implemented

Repo concept	Current state	Gap to repo treatment
Task Decomposition (01-core-cognitive/planning/task-decomposition.md)	agents/run-rebranding.ts runs a fixed sequential pipeline (text !' verify !' repackage).	Static hard-coded ordering, no dynamic/runtime decomposition of an arbitrary goal.
Supervisor / Coordinator (06-multi-agent/README.md#supervisor-pattern)	run-rebranding.ts orchestrates worker agents.	Linear script, not a supervisor that routes/monitors/synthesizes dynamically.

Repo concept	Current state	Gap to repo treatment
Manager–Worker (06-multi-agent/README.md#manager-worker)	Same orchestrator !’ agents.	No per-sub-task specialization or integration step beyond fixed order.
Delegation Patterns (06-multi-agent/README.md#delegation)	Orchestrator hands the whole context to each agent in turn.	No scoped task assignment, success criteria, or result-return contract per delegate.
Critic / Judge (06-multi-agent/README.md#critic-judge)	verification-agent.ts is a deterministic critic.	Not an independent LLM judge; no arbitration over another agent’s reasoning.
Agent Communication Protocols (06-multi-agent/README.md#agent-communication)	Proprietary JSON (hermes_bridge.py log lines, agents-live.json feed).	Ad-hoc schema, not a formal inter-agent message/handoff protocol.
Plan-and-Execute (13-agent-patterns/plan-and-execute.md)	Rebranding flow is plan-then-execute.	Plan is static; no replan-on-failure loop.
Verification / Self-Consistency (04-decision-making/README.md#verification)	The verification half is strongly implemented: agents/verification-agent.ts checks output against ground truth (expected logos, remaining references) and reports.	The self-consistency half (sampling multiple answers, voting) is absent; single deterministic pass only.
Confidence Estimation (04-decision-making/README.md#confidence-estimation)	research-agent.md output schema has a confidence_score field.	Spec-only field; nothing computes, calibrates, or gates on it.
Prompt Engineering (03-communication/README.md#prompt-engineering)	hermes_bridge.py forwards a prompt per node.	No systematic prompt design, versioning, or evaluation.
Research Agents (05-domain-skills/README.md#research)	agents/research-agent.md spec (Tavily, parallel search, citations).	Spec document only, no running implementation.
Research Agent Workflow (18-workflows/README.md#research-agent)	agents/research-agent.md spec + agents/research-results/ output folder.	Spec + artifacts only; no plan!’retrieve!’synthesize!’cite pipeline running.
Web Search Tools (02-tool-use/README.md#web-search)	agents/research-agent.md spec names Tavily Search API.	Spec-only tool choice; no web-search tool wired into a running agent.
Multimodal (05-domain-skills/README.md#multimodal)	visual-design-agent.md spec, agents/image-scraping-agent/, vector stack embeds image/video.	Specs + ingestion plan; no end-to-end multimodal agent.

Repo concept	Current state	Gap to repo treatment
Chunking (10-rag/chunking.md)	Vector stack describes ingestion (Vision-LLM description per asset).	Planned (doc status “Awaiting Approval”), not confirmed running.
Embeddings (10-rag/embeddings.md)	Vector stack: sentence-transformers !’ Vespa.	Planned in architecture, implementation unconfirmed.
GraphRAG (10-rag/advanced-rag.md#graphrag)	Vector stack has a full GraphRAG section (Neo4j + Vespa merge).	Architected on paper; no confirmed running GraphRAG query path.
MCP Protocol / Servers (11-mcp/protocol.md, 11-mcp/servers.md)	mcp-servers/playwright-server, .mcp.json present.	MCP is wired for browser automation; not broadly adopted across the army.
Browser Automation (02-tool-use/README.md#browser-automation)	Playwright MCP server available.	Available as a server; no agent confirmed driving it end-to-end.
API Calling (02-tool-use/README.md#api-calling)	agent-backend/providers/, hermes_bridge.py model calls.	Provider plumbing exists; no standardized agent API-calling pattern.
Database Access (02-tool-use/README.md#database-access)	agent-backend reads/writes Postgres (fleet schema).	Control-plane DB access, not exposed as an agent tool.
Tracing (14-observability/README.md#tracing)	fleet_telemetry + fleet_activity + agents-live.json.	Activity/telemetry rows exist; no correlated end-to-end trace per task.
Latency/Cost/Quality Tradeoffs (15-evaluation/README.md#tradeoffs)	fleet_telemetry collects cost_usd + runtime_sec.	Raw metrics captured; no systematic tradeoff analysis or decisioning.
Model Comparison (17-models/README.md)	fleet_agents.model (default qwen3.6:latest), provider (hermes).	Model is configurable per agent; no comparison/selection framework.
Risk Analysis (04-decision-making/README.md#risk-analysis)	Protocol classifies destructive ops as requiring backup/approval.	Risk gating is file/command-level policy, not a per-decision risk model.
Permissions & Least Privilege (07-safety-alignment/README.md#permissions)	Hard-protected file list scopes what may not be touched.	Guards files, not per-agent tool scopes/credentials.
Feedback Loops (08-learning-adaptation/README.md#feedback-loops)	hermes_bridge.py emits progress; verification reports findings.	No closed loop that feeds outcomes back to change future behavior.

Ø=Ý5 New ideas not implemented

Each item cites the repo page(s) it came from. Grouped by theme; the numbered headings map to the plan's preliminary findings.

1. Advanced reasoning patterns

- Tree of Thought — `reference/ai-agent-skills/01-core-cognitive/reasoning/tree-of-thought.md`. Explore multiple reasoning branches with evaluation + backtracking. Current agents are single-pass/deterministic.
- Graph of Thought — `reference/ai-agent-skills/01-core-cognitive/reasoning/graph-of-thought.md`. Merge/refine partial solutions (multi-parent nodes), which trees can't express.
- Chain of Thought — `reference/ai-agent-skills/01-core-cognitive/reasoning/chain-of-thought.md`. No explicit step-by-step reasoning prompting in the army.
- Self-Discover — `reference/ai-agent-skills/13-agent-patterns/self-discover.md`. Model selects/adapts/composes its own reasoning structure from a module library per task.
- Self-Reflection / Self-Correction — `reference/ai-agent-skills/01-core-cognitive/reasoning/self-reflection.md`. Agent critiques and revises its own output; distinct from the external `verification-agent`.
- Hierarchical Planning — `reference/ai-agent-skills/01-core-cognitive/planning/README.md`. Multi-level goal!'subgoaþlanning.
- ReAct — `reference/ai-agent-skills/13-agent-patterns/react.md`. Interleaved reason!'act!'observeloop.
- CodeAct — `reference/ai-agent-skills/13-agent-patterns/codeact.md`. Actions expressed as executable code in a sandbox.
- Voyager — `reference/ai-agent-skills/13-agent-patterns/voyager.md`. Open-ended skill-library accumulation.

2. Reflexion loop

- Reflexion — `reference/ai-agent-skills/13-agent-patterns/reflexion.md`. Persist a verbal reflection on why an attempt failed and inject it into the next attempt (gradient-free learning). `verification-agent.ts` is a critic, but there is no failure!'reflect!'store!'retryloop.

3. Advanced RAG

- Hybrid Search — `reference/ai-agent-skills/10-rag/retrieval-strategies.md#hybrid-search` . Fuse dense + sparse (BM25) via reciprocal rank fusion.
- Reranking — `reference/ai-agent-skills/10-rag/retrieval-strategies.md#reranking`. Cross-encoder second-stage re-scoring of top-N candidates.
- CRAG (Corrective RAG) — `reference/ai-agent-skills/10-rag/advanced-rag.md#crag`. Grade retrieved docs; refine or fall back to web search when insufficient.
- Self-RAG — `reference/ai-agent-skills/10-rag/advanced-rag.md#self-rag`. Decide when to retrieve, critique passages, and check groundedness via reflection tokens.
- (GraphRAG is adjacent — architected in the vector stack plan but unimplemented.)

4. Production memory systems

- Working / Long-Term / Semantic / Episodic Memory — `reference/ai-agent-skills/01-core-cognitive/memory/README.md` and `reference/ai-agent-skills/12-memory/README.md`. Storage choices, retrieval integration, write/retention policies. Agents are stateless per run today.
- Memory Compression — `reference/ai-agent-skills/01-core-cognitive/memory/README.md#memory-compression`. Summarize/decay old memories to bound storage.

5. Multi-agent patterns beyond supervisor

- Debate Pattern — `reference/ai-agent-skills/06-multi-agent/README.md#debate`. Multiple agents argue positions; a judge/vote resolves.
- Swarm Pattern — `reference/ai-agent-skills/06-multi-agent/README.md#swarm`. Peer-to-peer/emergent coordination (surveyed conceptually in `agentarmy.html`, not implemented).
- Formal delegation protocols — `reference/ai-agent-skills/06-multi-agent/README.md#delegation`. Scoped task assignment with success criteria and result return between `agents/*`.

6. Safety depth

- Prompt Injection Defense — `reference/ai-agent-skills/07-safety-alignment/README.md#prompt-injection`. Treat external content as untrusted data; never grant it instruction authority.

- Jailbreak Resistance — `reference/ai-agent-skills/07-safety-alignment/README.md#jailbreaks`. Defense-in-depth beyond model judgment.
- Least-privilege tool permissions — `reference/ai-agent-skills/07-safety-alignment/README.md#permissions`. Per-agent tool/credential scoping (current protocol guards files, not tool scopes).

7. Confidence, fallback, hallucination

- Fallback Strategies — `reference/ai-agent-skills/04-decision-making/README.md#fallback-strategies`. Retry / escalate / decline on low confidence instead of guessing.
- Hallucination Detection — `reference/ai-agent-skills/04-decision-making/README.md#hallucination-detection`. Grounding checks, self-consistency, citation verification.
- Confidence gating — `reference/ai-agent-skills/04-decision-making/README.md#confidence-estimation`. Act only when calibrated confidence is sufficient (no gating exists today).

8. Evaluation harness

- LLM-as-a-Judge — `reference/ai-agent-skills/15-evaluation/README.md#llm-as-judge`. Rubric-based judge model with known-bias mitigations.
- Benchmarks — `reference/ai-agent-skills/15-evaluation/README.md#benchmarks`. Custom/domain task sets + regression sets re-run on change.
- Latency/Cost/Quality tradeoff analysis — `reference/ai-agent-skills/15-evaluation/README.md#tradeoffs`. Systematic decisioning over the telemetry already collected.

9. Structured inter-agent handoffs

- Schema-enforced tool calls / handoffs — `reference/ai-agent-skills/03-communication/README.md#structured-outputs` and `reference/ai-agent-skills/06-multi-agent/README.md#agent-communication`. Enforce validated schemas at each agent boundary (relevant to the `agents-live.json` feed and `hermes_bridge.py` output).

10. Ready-made workflow blueprints

- Customer Support Agent — `reference/ai-agent-skills/18-workflows/README.md#customer-support`. Maps to SmartChannel CX.
- Email Agent — `reference/ai-agent-skills/18-workflows/README.md#email-agent`. Maps to VidiMail (human approval before send).
- Coding Agent — `reference/ai-agent-skills/18-workflows/README.md#coding-agent`. Write!'run!'reflect loop.
- Recruiter Agent — `reference/ai-agent-skills/18-workflows/README.md#recruiter-agent`. Screening support with humans in the loop.
- Legal Assistant — `reference/ai-agent-skills/18-workflows/README.md#legal-assistant`. RAG + strict citation/grounding.
- Data Analyst Agent — `reference/ai-agent-skills/18-workflows/README.md#data-analyst-agent`. Code execution + verified numerics.

11. Remaining catalog gaps (not in the plan's preliminary list)

- Function / Tool Calling — `reference/ai-agent-skills/02-tool-use/README.md#function-calling`. Formal LLM tool-call schema (agents are code-orchestrated, not LLM tool-calling).
- CLI / Shell Tools — `reference/ai-agent-skills/02-tool-use/README.md#cli-tools`.
- Python / Code Execution (as an agent action) — `reference/ai-agent-skills/02-tool-use/README.md#code-execution`. Sandboxed execution as a tool.
- Summarization — `reference/ai-agent-skills/03-communication/README.md#summarization`.
- Translation — `reference/ai-agent-skills/03-communication/README.md#translation`.
- Conversation State Management — `reference/ai-agent-skills/03-communication/README.md#conversation-state`.
- Coding Agents (domain) — `reference/ai-agent-skills/05-domain-skills/README.md#coding`.
- Finance Agents — `reference/ai-agent-skills/05-domain-skills/README.md#finance`.
- Medical Agents — `reference/ai-agent-skills/05-domain-skills/README.md#medicine`.
- Few-Shot Learning — `reference/ai-agent-skills/08-learning-adaptation/README.md#few-shot`.
- In-Context Learning — `reference/ai-agent-skills/08-learning-adaptation/README.md#in-context-learning`.
- Knowledge Updating — `reference/ai-agent-skills/08-learning-adaptation/README.md#knowledge-updating`.
- MCP Clients — `reference/ai-agent-skills/11-mcp/clients.md`.

- MCP Tools/Resources/Prompts primitives — `reference/ai-agent-skills/11-mcp/primitives.md`.
- MCP Auth & Transport — `reference/ai-agent-skills/11-mcp/security-and-transport.md`.
- Kubernetes for Agents — `reference/ai-agent-skills/16-deployment/README.md#kubernetes`.
- Serverless Agents — `reference/ai-agent-skills/16-deployment/README.md#serverless`.
- Edge Deployment — `reference/ai-agent-skills/16-deployment/README.md#edge`.
- Recipes / Cookbook — `reference/ai-agent-skills/19-recipes/README.md`.
- Case Studies — `reference/ai-agent-skills/20-case-studies/README.md` (`agentarmy.html` is a platform survey, not production case studies).

Summary counts

Counted against the 87 rows of `SKILL_CATALOG.md` (the MCP Protocol + MCP Servers rows share one Ø=ßá row above):

Bucket	Catalog rows
' Implemented	8
Ø=ßá Partial	28
Ø=Ý5 New / not implemented	51

Some Ø=Ý5 bullets cover several catalog rows at once (e.g. the memory bullet covers working/long-term/semantic/episodic); a few bullets in groups 5–9 deepen an existing Ø=ßá (delegation protocols, confidence gating, tradeoff analysis, schema-enforced handoffs, tool-scope permissions) rather than adding a new catalog row. Group 10 additionally lists three workflow blueprints (recruiter, legal, data analyst) that appear in `18-workflows/README.md` but not in the catalog table.

Highest-leverage adoptions (recommended follow-up, out of scope here)

1. Reflexion loop (#2) + Self-Reflection (#1) — cheapest way to make the existing critic (`verification-agent.ts`) actually improve outcomes across retries.
2. Hybrid Search + Reranking (#3) — incremental upgrades on the already-running Vespa stack, before jumping to CRAG/Self-RAG.

3. Confidence gating + Fallback (#7) — turns the already-specced `confidence_score` field into a real decision gate.
4. LLM-as-a-Judge + regression benchmark (#8) — reuses `fleet_telemetry` data to prove quality across changes.
5. Prompt-injection defense + least-privilege tool scopes (#6) — extends the file-guardrail protocol to tool/content-level safety.

Licensing note: the vendored snapshot is MIT (code) + CC BY 4.0 (prose). Internal reference use is fine; if any of this content is republished on VidiSmart pages, attribute `github.com/sreerevanth/AI-Agent-Skills`.